

Android (Kotlin) + Firebase (Firestore)

1. Crie um projeto e altere o SDK para 37.
2. Configure o Firebase (como ensinado em aula).
3. No **build.gradle.kts (módulo:app)**, adicione as dependências:

```
implementation("com.google.firebase:firebase-firestore")
implementation("androidx.compose.material:material-icons-extended:1.7.8")
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.6")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-play-services:1.9.0")
```

4. Vamos criar a camada **model**, ela ficará dentro da camada **data**. Sua estrutura será composta por:

```
import com.google.firebase.firestore.DocumentId
```

```
data class Pessoa(
    @DocumentId
    val id: String = "",
    val nome: String = "",
    val cidade: String = ""
)
```

5. Agora vamos criar a camada **repository**, ela ficará dentro da camada **data**. Sua estrutura será composta por:

```
import br.com.treinamento.projetocrud.data.model.Pessoa
import com.google.firebase.firestore.FirebaseFirestore
import kotlinx.coroutines.tasks.await

class PessoaRepository {

    private val db = FirebaseFirestore.getInstance()
    private val collection = db.collection("pessoas")

    // READ - busca todas as pessoas cadastradas
    suspend fun getPessoas(): List<Pessoa> {
        val snapshot = collection.get().await()
        return snapshot.toObject(Pessoa::class.java)
    }

    // CREATE - adiciona uma nova pessoa (o Firestore gera o id automaticamente)
    suspend fun addPessoa(pessoa: Pessoa) {
        collection.add(pessoa).await()
    }

    // UPDATE - sobrescreve o documento existente usando o id da pessoa
    suspend fun updatePessoa(pessoa: Pessoa) {
        collection.document(pessoa.id).set(pessoa).await()
    }

    // DELETE - remove o documento pelo id
    suspend fun deletePessoa(id: String) {
        collection.document(id).delete().await()
    }
}
```

6. Agora podemos criar a camada **viewmodel**, e implementar essa estrutura:

// Representa tudo o que a tela precisa saber sobre o estado atual

```
data class PessoaUiState(  
    val pessoas: List<Pessoa> = emptyList(),  
    val isLoading: Boolean = false,  
    val errorMessage: String? = null,  
    val pessoaEmEdicao: Pessoa? = null // null = criando nova; não-null = editando  
)
```

```
class PessoaViewModel(  
    private val repository: PessoaRepository = PessoaRepository()  
) : ViewModel() {  
  
    private val _uiState = MutableStateFlow(PessoaUiState())  
    val uiState: StateFlow<PessoaUiState> = _uiState.asStateFlow()  
  
    init {  
        carregarPessoas()  
    }  
  
    fun carregarPessoas() {  
        viewModelScope.launch {  
            _uiState.value = _uiState.value.copy(isLoading = true, errorMessage = null)  
            try {  
                val lista = repository.getPessoas()  
                _uiState.value = _uiState.value.copy(pessoas = lista, isLoading = false)  
            } catch (e: Exception) {  
                _uiState.value = _uiState.value.copy(  
                    isLoading = false,  
                    errorMessage = "Erro ao carregar: ${e.message}"  
                )  
            }  
        }  
    }  
}
```

```

fun salvarPessoa(nome: String, cidade: String) {
    viewModelScope.launch {
        try {
            val emEdicao = _uiState.value.pessoaEmEdicao
            if (emEdicao == null) {
                repository.addPessoa(Pessoa(nome = nome, cidade = cidade))
            } else {
                repository.updatePessoa(emEdicao.copy(nome = nome, cidade = cidade))
            }
            _uiState.value = _uiState.value.copy(pessoaEmEdicao = null)
            carregarPessoas() // recarrega a lista após salvar
        } catch (e: Exception) {
            _uiState.value = _uiState.value.copy(errorMessage = "Erro ao salvar:
${e.message}")
        }
    }
}

fun selecionarParaEdicao(pessoa: Pessoa) {
    _uiState.value = _uiState.value.copy(pessoaEmEdicao = pessoa)
}

fun cancelarEdicao() {
    _uiState.value = _uiState.value.copy(pessoaEmEdicao = null)
}

fun excluirPessoa(id: String) {
    viewModelScope.launch {
        try {
            repository.deletePessoa(id)
            carregarPessoas()
        } catch (e: Exception) {

```

```

        _uiState.value = _uiState.value.copy(errorMessage = "Erro ao excluir:
        ${e.message}")
    }
}
}
}

```

7. Vamos para a última camada, crie o pacote **view**:

```

@Composable
fun PessoaScreen(viewModel: PessoaViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsState()

    var nome by remember { mutableStateOf("") }
    var cidade by remember { mutableStateOf("") }

    // Sempre que o item "em edição" mudar, preenchemos os campos do formulário
    LaunchedEffect(uiState.pessoaEmEdicao) {
        nome = uiState.pessoaEmEdicao?.nome ?: ""
        cidade = uiState.pessoaEmEdicao?.cidade ?: ""
    }

    Column(
        modifier = Modifier
            .safeDrawingPadding()
            .fillMaxSize()
            .padding(16.dp)
    ) {
        Text(
            text = if (uiState.pessoaEmEdicao == null) "Nova pessoa" else "Editando
            pessoa",
            style = MaterialTheme.typography.titleMedium
        )
    }
}

```

```
Spacer(modifier = Modifier.height(8.dp))
```

```
OutlinedTextField(  
    value = nome,  
    onValueChange = { nome = it },  
    label = { Text("Nome") },  
    modifier = Modifier.fillMaxWidth()  
)
```

```
Spacer(modifier = Modifier.height(8.dp))
```

```
OutlinedTextField(  
    value = cidade,  
    onValueChange = { cidade = it },  
    label = { Text("Cidade") },  
    modifier = Modifier.fillMaxWidth()  
)
```

```
Spacer(modifier = Modifier.height(8.dp))
```

```
Row {  
    Button(  
        onClick = {  
            if (nome.isNotBlank() && cidade.isNotBlank()) {  
                viewModel.salvarPessoa(nome, cidade)  
                nome = ""  
                cidade = ""  
            }  
        }) {  
        Text(if (uiState.pessoaEmEdicao == null) "Adicionar" else "Salvar alterações")  
    }  
}
```

```

        if (uiState.pessoaEmEdicao != null) {
            Spacer(modifier = Modifier.width(8.dp))
            OutlinedButton(onClick = { viewModel.cancelarEdicao() }) {
                Text("Cancelar")
            }
        }
    }

    Spacer(modifier = Modifier.height(16.dp))

    if (uiState.isLoading) {
        CircularProgressIndicator()
    }

    uiState.errorMessage?.let { mensagem ->
        Text(text = mensagem, color = MaterialTheme.colorScheme.error)
    }

    LazyColumn(modifier = Modifier.fillMaxWidth()) {
        items(uiState.pessoas, key = { it.id }) { pessoa ->
            PessoaItem(
                pessoa = pessoa,
                onEditClick = { viewModel.selecionarParaEdicao(pessoa) },
                onDeleteClick = { viewModel.excluirPessoa(pessoa.id) }
            )
        }
    }
}

```

```

@Composable
fun PessoaItem(
    pessoa: Pessoa,
    onEditClick: () -> Unit,

```

```

        onDeleteClick: () -> Unit
    ){
        Card(
            modifier = Modifier
                .fillMaxWidth()
                .padding(vertical = 4.dp)
        ){
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(12.dp),
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.SpaceBetween
            ){
                Column {
                    Text(text = pessoa.nome, style = MaterialTheme.typography.bodyLarge)
                    Text(text = pessoa.cidade, style = MaterialTheme.typography.bodySmall)
                }
                Row {
                    IconButton(onClick = onEditClick) {
                        Icon(Icons.Default.Edit, contentDescription = "Editar")
                    }
                    IconButton(onClick = onDeleteClick) {
                        Icon(Icons.Default.Delete, contentDescription = "Excluir")
                    }
                }
            }
        }
    }
}

```

8. No arquivo **MainActivity**, adicione o **PersonScreen()**.
9. Teste a aplicação.